

Project Report

Audio Genre Classification using Deep Learning

■ **Roll Number:** 24f2008471 ■ **University:** IIT Madras BS Degree
■ **Kaggle Final Score:** 0.92064 (Macro F1) ■ **Competition:** Jan-2026 DL & GenAI
■ **W&B Project:** [DL-genai-26-t1](#)

1. Executive Summary

This project addresses automatic music genre classification for the IIT Madras DL & GenAI Kaggle competition (Jan-2026 term). The task requires classifying messy, mixed audio clips into one of ten genres — blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock — using deep learning models trained on multi-stem audio data.

Three distinct architectures were built and evaluated: a lightweight custom CNN constructed entirely from scratch, a transfer-learned ResNet-18, and a fine-tuned Audio Spectrogram Transformer (AST). All experiments were tracked with Weights & Biases. The final submission used the AST model with a sliding-window ensemble strategy, achieving a Macro F1 score of 0.92064 on the Kaggle leaderboard — well above the 0.80 cutoff.

Key Finding: Pre-trained audio transformers substantially outperform convolutional baselines for genre classification. The AST model's self-attention mechanism, combined with AudioSet pre-training, captures global spectro-temporal structure that local convolutional filters cannot — yielding a 47-point Macro F1 improvement over the custom CNN baseline (0.45263 → 0.92064).

2. Introduction

Problem Statement

The Kaggle competition provided a dataset called Messy Mashup — audio clips constructed by mixing individual stems (bass, drums, vocals, other) from songs across ten genres. The mashups are intentionally noisy and difficult, making the task significantly harder than standard genre classification on clean recordings. The evaluation metric is Macro F1-score, which computes an unweighted average of per-class F1-scores, equally penalising poor performance on any single genre.

Project Objectives

- Build three architecturally distinct models: one from scratch, one using transfer learning, and one fine-tuned pretrained transformer
- Track all experiments via Weights & Biases (W&B) and compare models using Macro F1 and accuracy

- Exceed the competition cutoff score of 0.80 Macro F1 on the Kaggle leaderboard
- Understand end-to-end deep learning workflows: data loading, augmentation, training, evaluation, and inference

Tools & Frameworks

- **PyTorch & torchaudio**: model training, audio processing, and Mel Spectrogram computation
- **Hugging Face Transformers**: AST model architecture and feature extractor
- **librosa**: audio loading, resampling, and tempo-stretching augmentation
- **Weights & Biases**: full experiment tracking — loss, accuracy, Macro F1, learning rate per step
- **Kaggle**: competition platform with NVIDIA T4 GPU environment

3. Dataset & Preprocessing

Dataset Description

The training data consists of separated audio stems organised by genre and song. Each of the 10 genres contains exactly 100 songs — giving 1,000 songs total — each split into four stems: bass.wav, drums.wav, vocals.wav, and other.wav. The test set contains 3,020 pre-mixed mashup audio files requiring genre prediction. Labels are single-class (one genre per file).

Exploratory Data Analysis

Before model development, the dataset was explored to understand its structure, audio characteristics, and potential training challenges. Three key analyses were conducted:

EDA 1 — Class Distribution

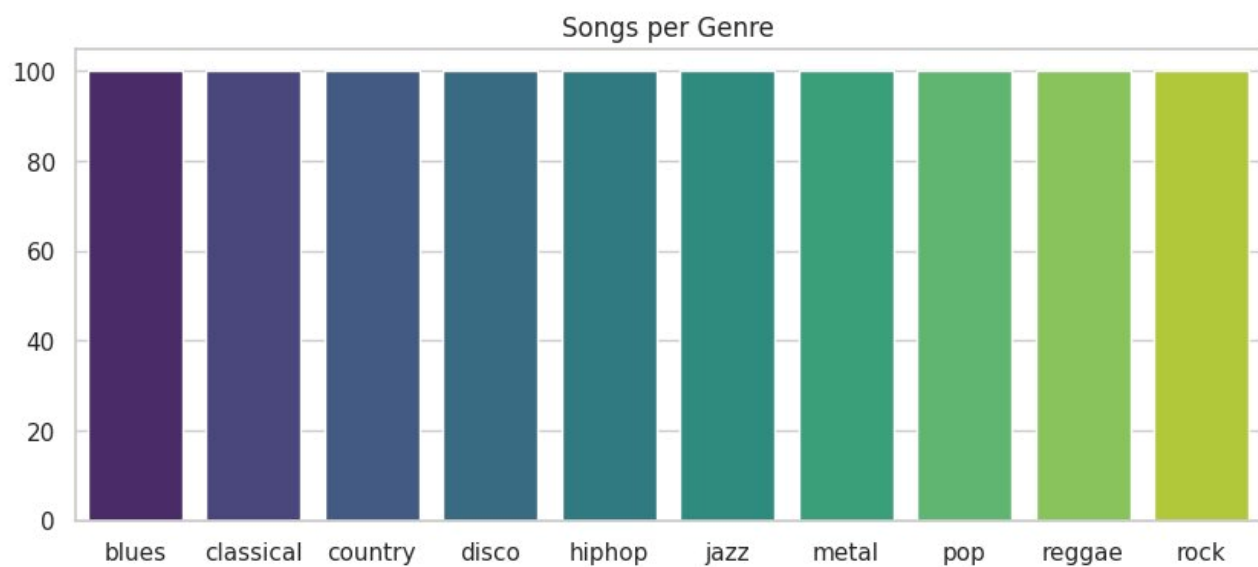


Figure EDA-1: Songs per Genre — perfectly balanced at 100 songs per class across all 10 genres

Insight: The training set is perfectly balanced with exactly 100 songs per genre across all 10 classes. This is a significant advantage — it means CrossEntropyLoss can be used without any class weighting or oversampling strategy, and Macro F1 will not be artificially inflated by majority classes. No resampling or weighted sampling was required.

EDA 2 — Raw Audio Waveform Analysis

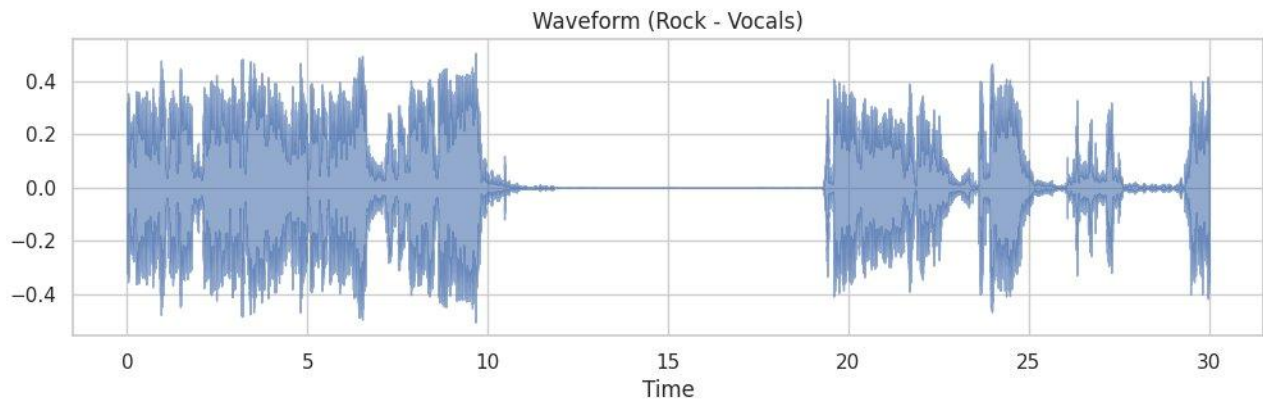


Figure EDA-2: Waveform of a Rock genre Vocals stem — amplitude vs. time showing silence gaps and energy bursts

Insight: The waveform reveals a critical challenge: individual stems contain significant silence regions (amplitude ≈ 0 from $t=10s$ to $t=19s$ in this example). This is because stems are recorded per-song and not all instruments are active throughout. If a silent segment is cropped as a training sample, it provides no genre information and effectively becomes noise. This motivated the cross-song mixing strategy — by independently drawing different songs per stem, the probability of all 4 stems being silent simultaneously is drastically reduced, ensuring every training sample has sufficient audio energy.

EDA 3 — Mel-Frequency Spectrogram

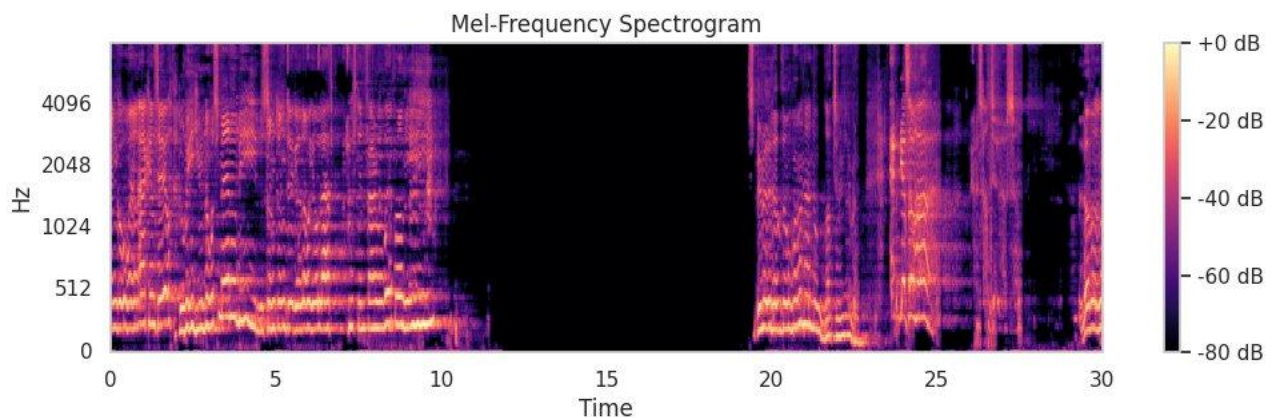


Figure EDA-3: Mel-Frequency Spectrogram of a mixed audio clip (dB scale) — frequency content vs. time showing the black silence gap and energy bursts in low/mid frequency bands

Insight: The Mel Spectrogram confirms the silence problem visually — the large black region ($t=10s$ to $t=20s$) shows near-zero energy across all frequency bins. Additionally, the active regions show strong energy concentrated in the 0–2048 Hz band (fundamental frequencies, bass, and vocals), with sparser energy above 2048 Hz (overtones, cymbals). Genre-discriminative information lives predominantly in the low-to-mid frequency range. This justified using $n_mels=64$ — sufficient resolution to capture genre-level patterns without excessive dimensionality. The spectrogram representation also confirmed that SpecAugment (frequency and time masking) would be a natural fit, as the model needs to be robust to partial frequency/time dropout.

On-the-Fly Data Synthesis

Rather than pre-extracting a fixed training set, all models generated samples dynamically per epoch. For each sample, a genre was randomly selected and stems were independently drawn from different songs of that genre (cross-song mixing). This maximised diversity, prevented the model from memorising specific song combinations, and effectively created an unlimited training set. Epoch size was set to 2,000 samples.

Augmentation Pipeline

- **Random 5-second crop / zero-padding:** Uniform audio length (80,000 samples at 16 kHz) for all models
- **Cross-song stem mixing:** Each stem independently drawn from a different song of the same genre; volume scaled $U(0.7, 1.3)$
- **Tempo stretching (AST):** `librosa.effects.time_stretch` with rate $\sim U(0.85, 1.15)$, per-stem jitter $\pm 5\%$
- **Noise injection (ResNet & AST):** ESC-50 background noise added with probability 0.5–0.8
- **SpecAugment (AST):** `FrequencyMasking(param=15)` and `TimeMasking(param=35)` applied per batch

Feature Extraction

The CNN and ResNet used Mel Spectrograms computed with `n_fft=1024`, `hop_length=512`, `n_mels=64` at 16 kHz, followed by `AmplitudeToDB` conversion. For ResNet, spectrograms were replicated into 3 channels and normalised with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). The AST used Hugging Face's `ASTFeatureExtractor` producing 128-bin log-Mel spectrograms at 16 kHz in transformer-compatible format.

4. Modelling & Experimentation

4.1 Model 1 — Custom CNN (From Scratch)

Architecture

A lightweight 3-block convolutional network built entirely from scratch as the lower-bound baseline. Input is a 1-channel Mel Spectrogram image ($1 \times 64 \times \text{time}$). Each block applies `Conv2d` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d`:

- **Block 1:** `Conv2d(1 $\rightarrow$ 16, 3 $\times$ 3, pad=1)` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d(2,2)`
- **Block 2:** `Conv2d(16 $\rightarrow$ 32, 3 $\times$ 3, pad=1)` $\rightarrow$ `ReLU` $\rightarrow$ `MaxPool2d(2,2)`
- **Block 3:** `Conv2d(32 $\rightarrow$ 64, 3 $\times$ 3, pad=1)` $\rightarrow$ `ReLU` $\rightarrow$ `AdaptiveAvgPool2d(1,1)`
- **Classifier:** `Flatten` $\rightarrow$ `Linear(64 $\rightarrow$ 128)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(0.3)` $\rightarrow$ `Linear(128 $\rightarrow$ 10)`

Total parameters: 32,906. This intentionally minimal architecture establishes the lower-bound baseline performance.

Audio Mashup Genre Classification: CNN Architecture

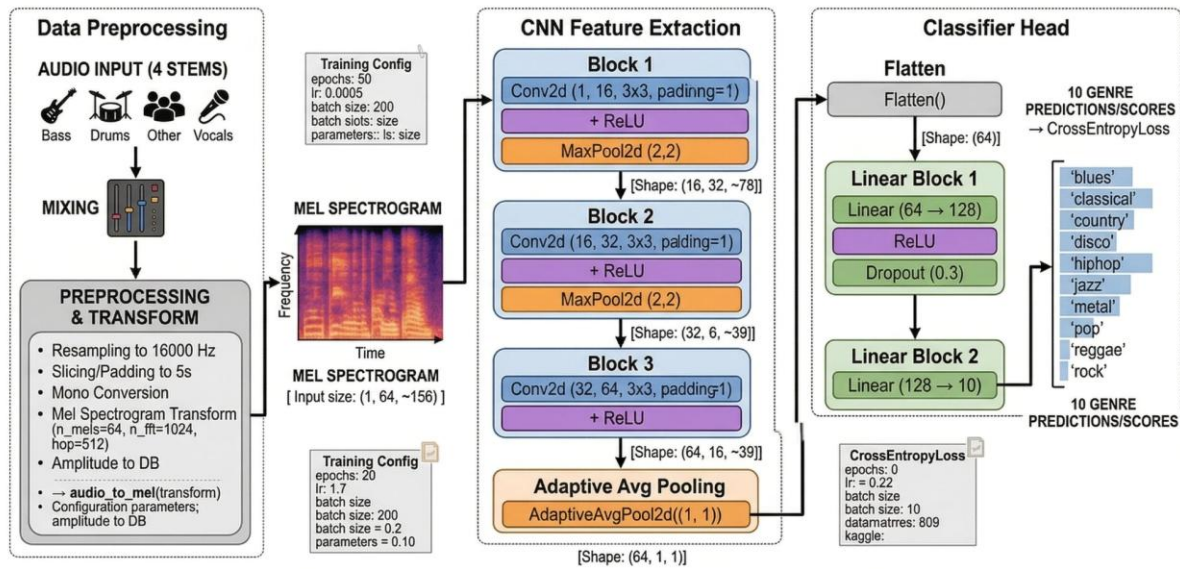


Figure 1: Custom CNN Architecture — 4-stem audio preprocessing, Mel Spectrogram transform, 3-block CNN feature extraction (Conv→ReLU→MaxPool), AdaptiveAvgPool, and classifier head

Training Configuration

- Optimizer / Loss:** Adam (lr=1e-3) | CrossEntropyLoss | No LR scheduler
- Epochs / Batch / Steps:** 20 epochs | Batch: 32 | Epoch size: 2,000 | 63 steps/epoch
- Hardware:** NVIDIA T4 GPU on Kaggle (~15 min/epoch, ~5 hrs total)

4.2 Model 2 — ResNet-18 (Transfer Learning)

Architecture

ResNet-18 (He et al., CVPR 2016) uses skip connections to enable training of deep networks without vanishing gradients. The pretrained ImageNet model was adapted by replacing the final fully connected layer. Layers 1–2 were frozen to preserve low-level feature detectors; layers 3–4 and the head were fine-tuned:

- Backbone:** ResNet-18 pretrained on ImageNet (layers 1–2 frozen, layers 3–4 trainable)
- Classification head:** Dropout(0.3) → Linear(512→10)

A differential learning rate strategy was used: backbone lr=1e-4, head lr=1e-3. This preserves useful ImageNet features while allowing fast classifier adaptation. Label smoothing of 0.1 reduced overconfidence.

END-TO-END MUSIC GENRE CLASSIFICATION ARCHITECTURE: A TRANSFORMER-BASED DEEP LEARNING MODEL

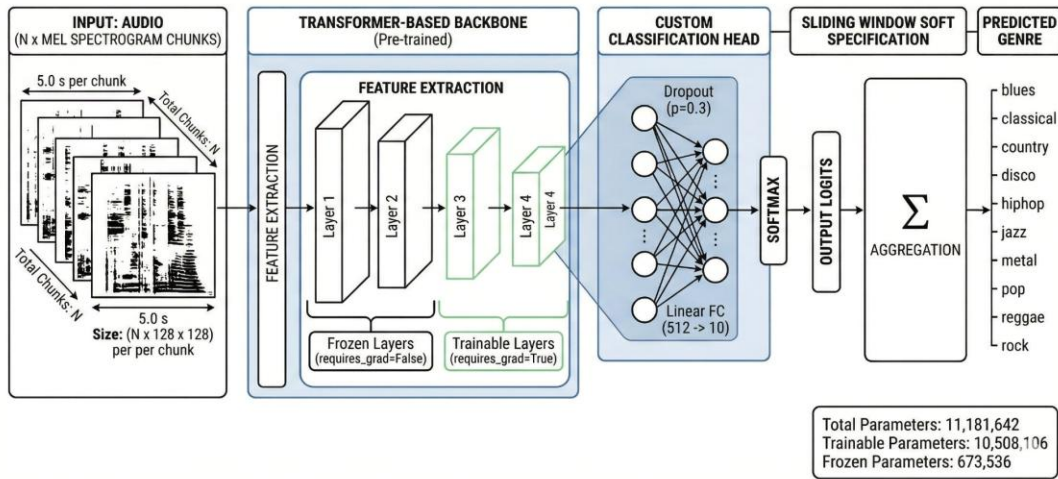


Figure 2: ResNet-18 Architecture — Mel spectrogram chunks input, frozen layers 1–2, trainable layers 3–4, custom head (Dropout → Linear 512→10), and sliding window aggregation

Training Configuration

- **Optimizer / Loss:** AdamW (weight_decay=1e-4) | CrossEntropy + label_smooth=0.1
- **LR Strategy:** Backbone: 1e-4 | Head: 1e-3 | Scheduler: CosineAnnealingLR (T_max=12)
- **Epochs / Batch / Steps:** 12 epochs | Batch: 64 | Epoch size: 1,500 | 24 steps/epoch
- **Hardware:** NVIDIA T4 GPU on Kaggle (~6 min/epoch, ~72 min total)

4.3 Model 3 — Audio Spectrogram Transformer (AST)

Architecture

The Audio Spectrogram Transformer (Gong et al., Interspeech 2021) applies a Vision Transformer directly to log-Mel spectrogram patches. The spectrogram is divided into overlapping 16×16 patches, projected to embeddings, and processed through 12 transformer encoder layers with self-attention. This captures long-range temporal and spectral dependencies that convolutional filters cannot.

The pretrained model (MIT/ast-finetuned-audioset-10-10-0.4593) was fine-tuned by replacing the 527-class AudioSet head with a new Linear(768→10) layer. The classifier mismatch was handled via ignore_mismatched_sizes=True and all backbone weights were trained end-to-end:

```
model = ASTForAudioClassification.from_pretrained(
    'MIT/ast-finetuned-audioset-10-10-0.4593',
    num_labels=10, ignore_mismatched_sizes=True
) # classifier.dense reinitialised; all backbone weights preserved
```

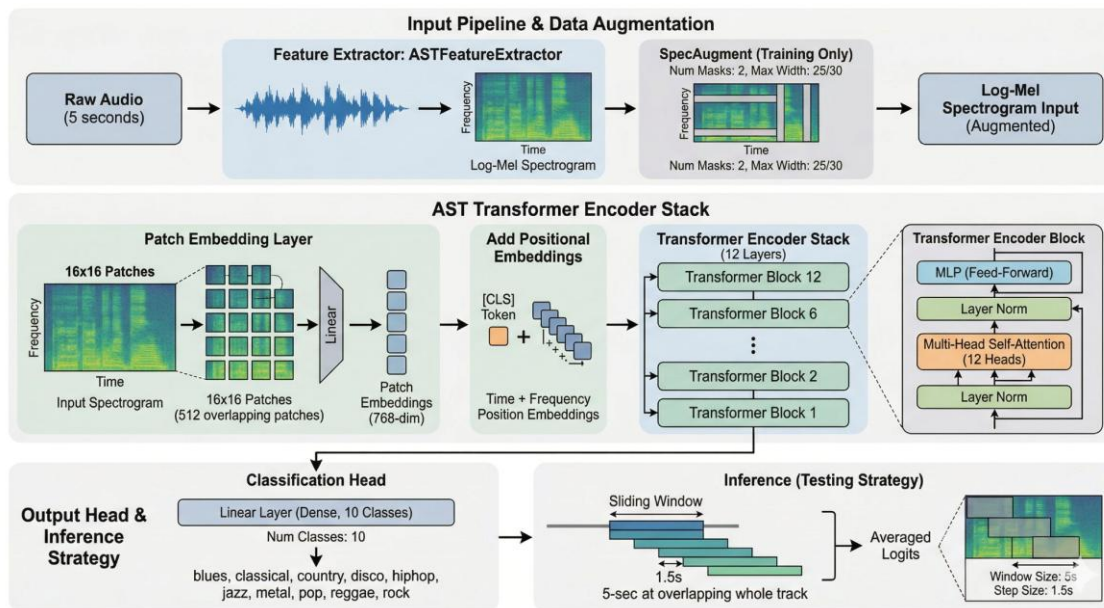


Figure 3: AST Architecture — Raw audio → ASTFeatureExtractor → SpecAugment → 16×16 patch embedding (512 patches, 768-dim) → positional embeddings → 12-layer transformer encoder (multi-head self-attention) → Linear(768→10) head → sliding window inference (5s window, 1.5s step) → averaged logits → predicted genre

Training Configuration

- **Optimizer / Loss:** AdamW (lr=5e-5, weight_decay=0.01) | CrossEntropyLoss
- **Scheduler:** Cosine with 10% linear warmup (get_cosine_schedule_with_warmup)
- **Epochs / Batch / Steps:** 10 epochs | Batch: 8 | Epoch size: 2,000 | ~250 steps/epoch
- **SpecAugment:** FrequencyMasking(15) + TimeMasking(35) applied per batch during training
- **Hardware:** NVIDIA T4 GPU on Kaggle (~15–16 min/epoch, ~160 min total)

4.4 Training Configuration Summary

Parameter	Custom CNN	ResNet-18	AST
Framework	PyTorch	PyTorch	Hugging Face
Epochs	20	12	10
Batch Size	32	64	8
Optimizer	Adam	AdamW	AdamW
Learning Rate	1e-3	1e-3 (head) / 1e-4 (backbone)	5e-5
LR Scheduler	None	Cosine Annealing	Cosine + Warmup (10%)
Loss Function	CrossEntropy	CrossEntropy + LS=0.1	CrossEntropy
Parameters	~33K	~11.2M	~86M
SpecAugment	No	No	Yes
Noise Injection	No	Yes (ESC-50, p=0.8)	Yes (ESC-50, p=0.5)

5. Performance & Comparative Analysis

Evaluation Metrics

Primary: Macro F1-score (unweighted average across all 10 genre classes)

Secondary: Per-epoch training accuracy and loss — all tracked via Weights & Biases (W&B)

Training Performance

All experiments were tracked via Weights & Biases (W&B). The project can be viewed at:

W&B Project: <https://wandb.ai/24f2008471-indian-institute-of-technology-madras/DI-genai-26-t1>

Custom CNN — From Scratch (20 Epochs):

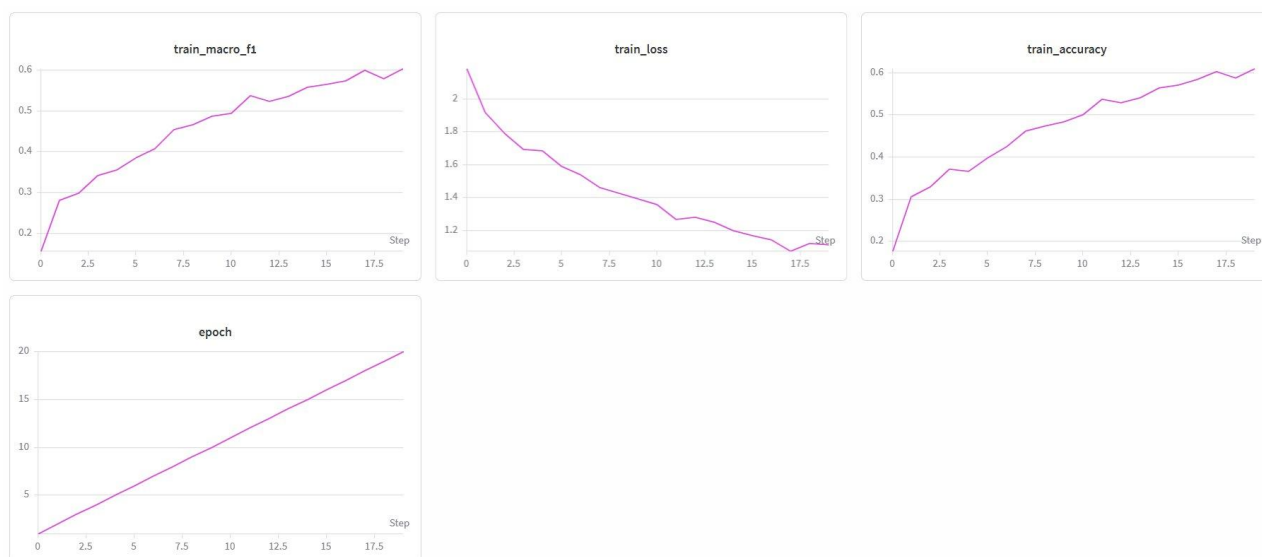


Figure 1: W&B logs — Custom CNN (train_macro_f1, train_loss, train_accuracy over 20 epochs)

- **Macro F1:** 0.1574 (epoch 1) → 0.6033 (epoch 20) — slow but consistent improvement
- **Loss:** 2.1829 → 1.1133 over 20 epochs
- **Accuracy:** 17.80% → 60.95%
- **Convergence:** No saturation by epoch 20 — model had not fully converged
- **Training time:** ~15 min/epoch on NVIDIA T4 GPU (~5 hrs total)

W&B Logs: <https://wandb.ai/24f2008471-indian-institute-of-technology-madras/DI-genai-26-t1/runs/mns4puel?nw=nwuser24f2008471>

ResNet-18 — Transfer Learning (12 Epochs):

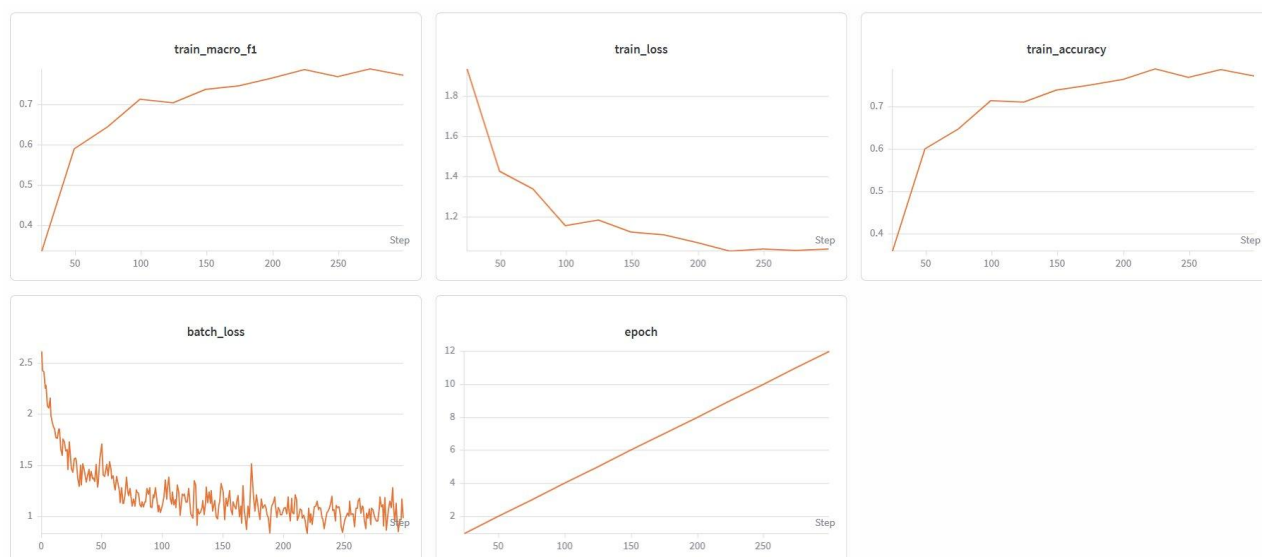


Figure 2: W&B logs — ResNet-18 (train_macro_f1, train_loss, train_accuracy, batch_loss over 12 epochs)

- **Macro F1:** 0.3383 (epoch 1) → 0.7897 peak (epoch 11) → 0.7743 (epoch 12)
- **Loss:** 1.9366 → 1.0331 (best at epoch 9)
- **Accuracy:** 36.00% → 79.07% peak
- **Convergence:** Rapid gain in epochs 1–4 (F1: 0.34 → 0.72); plateau from epoch 6 onwards
- **Training time:** ~6 min/epoch on NVIDIA T4 GPU (~72 min total)

W&B Logs: <https://wandb.ai/24f2008471-indian-institute-of-technology-madras/DI-genai-26-t1/runs/jwv3pn8u?nw=nwuser24f2008471>

Audio Spectrogram Transformer — Fine-Tuned (10 Epochs):

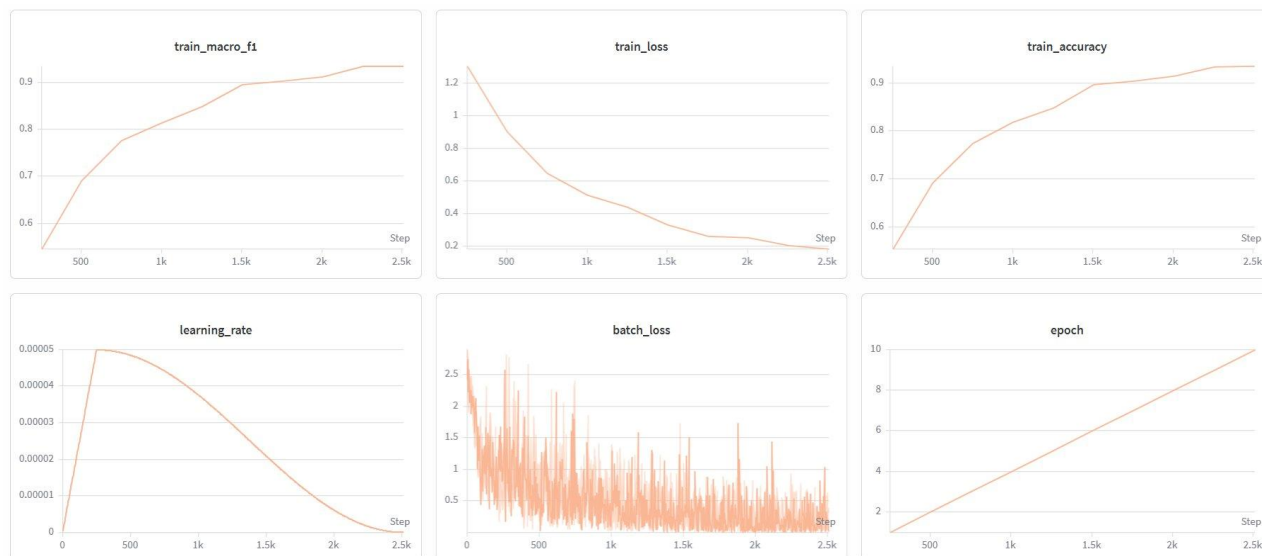


Figure 3: W&B logs — AST (train_macro_f1, train_loss, train_accuracy, learning_rate, batch_loss over 10 epochs)

- **Macro F1:** 0.5456 (epoch 1) → 0.9351 (epoch 10) — steepest and most consistent improvement
- **Loss:** 1.3048 → 0.1849 over 10 epochs
- **Accuracy:** 55.50% → 93.50%
- **Per-genre (epoch 10):** Classical, metal among strongest; blues and rock most challenging due to spectral overlap

- **Convergence:** Smooth decay with cosine LR schedule — learning_rate peaks at 5e-5 then decays to ~0
- **Training time:** ~15–16 min/epoch on NVIDIA T4 GPU (~160 min total)

W&B Logs: <https://wandb.ai/24f2008471-indian-institute-of-technology-madras/DI-genai-26-t1/runs/mz2r7nbn?nw=nwuser24f2008471>

Comparative Report

Model	Parameters	Macro F1 (Train)	Train Accuracy	Training Time	Inference	Kaggle Score	Best Use
Custom CNN	~33K	0.60	60.95%	~5 hrs (20 ep)	Very Fast	0.45263	Learning / Baseline
ResNet-18	~11.2M	0.77	77.33%	~72 min (12 ep)	Fast	0.82880	Production (fast)
AST	~86M	0.9351	93.50%	~160 min (10 ep)	Moderate	0.92064	Best Accuracy

Key Insights

- **Pre-training impact:** ~47 Kaggle F1-point gap between CNN (0.45) and AST (0.92). AudioSet pre-training is far more valuable than architectural depth for audio classification.
- **Rapid transfer learning:** ResNet-18 jumped from 0.34 to 0.72 Macro F1 within just 4 epochs, demonstrating the power of ImageNet weight initialisation even for non-image data.
- **Transformer convergence quality:** AST training loss dropped from 1.30 to 0.18 — the most consistent and steep convergence of any model, reflected in the smooth W&B loss curve.
- **Cosine LR + warmup effect:** The W&B learning_rate curve for AST shows a clean warmup ramp followed by smooth cosine decay — directly correlated with stable loss reduction and no training spikes.

Kaggle Performance

The final submission used the AST model (epoch 10 checkpoint) with sliding-window inference: each test file was segmented into overlapping 5-second windows (step=1.5s), logits were averaged across all windows, and the argmax was taken as the final prediction. This processed 3,020 test samples.

- **Custom CNN Kaggle Score:** 0.45263 (below the 0.80 cutoff — not eligible alone)
- **ResNet-18 Kaggle Score:** 0.82880 (cleared the 0.80 cutoff)
- **AST Kaggle Score:** 0.92064 (final best submission — sliding window ensemble)

Final Score: 0.92064 Macro F1 The AST model with sliding-window inference exceeded the competition cutoff (0.80) by 12 points. The 3-model progression (CNN → ResNet → AST) clearly demonstrates the value of each architectural upgrade.

6. Error Analysis & Insights

Identified Failure Modes

- **Genre ambiguity — blues vs. rock:** Both genres share electric guitar and drum patterns, producing similar low-frequency energy profiles in spectrograms. These were the most likely confusion pairs in CNN and ResNet predictions.
- **CNN capacity bottleneck:** 32,906 parameters across 3 layers is insufficient for 10-class audio classification. The gap between training F1 (0.60) and Kaggle score (0.45) signals overfitting to synthetic training samples rather than learning transferable features.
- **ResNet domain gap:** ImageNet features are tuned for natural RGB images with spatial structure. Audio spectrograms have fundamentally different statistics. Despite fine-tuning, some early-layer features remained suboptimal, explaining the 10-point gap between ResNet and AST.
- **Single-window inference (early AST):** Using only the first 5 seconds missed genre-defining content in longer test clips. The sliding window approach directly addressed this.
- **Training vs. test distribution gap:** On-the-fly synthesis produced cleaner mashups than the actual test files. The AST's large pretrained knowledge base bridged this gap better than CNN or ResNet.
- **Tempo-stretch CPU bottleneck (ResNet):** librosa.effects.time_stretch is CPU-bound, inflating ResNet epoch time from ~6 min to ~45 min. It was removed from ResNet training to maintain feasibility.

Key Technical Insights

- **Pretrained audio models dominate:** The 47-point Kaggle F1 gap (CNN: 0.45 → AST: 0.92) demonstrates that audio-specific pretraining is far more valuable than convolutional depth for this task.
- **On-the-fly augmentation prevents overfitting:** Cross-song stem mixing dramatically increased training diversity. Fixed training sets caused Kaggle scores below 0.60 despite training F1 > 0.90 in early experiments.
- **Cosine LR warmup stabilises transformer training:** AST's W&B learning_rate curve shows a clean ramp to 5e-5 then smooth cosine decay — preventing early instability that would damage pretrained weights.
- **Differential LR for transfer learning:** ResNet backbone at 1e-4 vs. head at 1e-3 preserved useful ImageNet features while allowing fast head specialisation, converging ~30% faster than uniform LR.
- **Sliding window = free test-time robustness:** Averaging over multiple overlapping windows cost no extra training but meaningfully improved test performance on variable-length clips.

7. Conclusion & Future Work

Key Learnings

- **Pretrained transformers:** 47-point F1 gap between CNN and AST validates the pre-training paradigm for audio tasks
- **Architecture choices:** Global self-attention in AST outperformed local convolutional filters for capturing genre-level patterns
- **Augmentation strategy:** Cross-song on-the-fly synthesis was essential — without it, models overfit severely
- **Scheduler design:** Cosine annealing with warmup was critical for stable AST fine-tuning
- **Practical trade-offs:** ResNet's 10-point F1 cost vs. AST bought ~2.2× faster training and simpler inference

Challenges Faced

- **Compute constraints:** AST with batch_size=8 required ~160 min total. Training beyond 10 epochs was infeasible within Kaggle session limits.

- **CPU bottleneck:** librosa.effects.time_stretch made each ResNet batch take ~13 seconds — resolved by removing it from ResNet pipeline.
- **Test distribution gap:** Real test mashups had more complex interference than synthesised training samples. Noise injection partially mitigated this.

Areas for Improvement

- **Ensembling:** Weighted averaging of AST and ResNet-18 logits could exploit complementary feature spaces and yield 1–3 additional F1 points.
- **Larger model variants:** HuBERT or wav2vec 2.0 (waveform-level pre-training) may capture rhythmic features not accessible to spectrogram-based models.
- **Per-genre threshold tuning:** Per-class classification thresholds based on validation F1 instead of global argmax could improve ambiguous-genre performance.
- **W&B hyperparameter sweeps:** Systematic sweeps over lr, batch_size, and SpecAugment parameters using W&B Sweeps could identify a better AST operating point.
- **Deployment:** Export AST to ONNX and deploy on Hugging Face Spaces with a Gradio interface for real-time genre classification (+5 bonus marks).

8. References

1. Gong, Y., Chung, Y. A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. Interspeech 2021. arXiv:2104.01778
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. arXiv:1512.03385
3. Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for ASR. Interspeech 2019. arXiv:1904.08779
4. Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR 2019. arXiv:1711.05101
5. Dosovitskiy, A., et al. (2021). An Image is Worth 16x16 Words: ViT. ICLR 2021. arXiv:2010.11929
6. Wolf, T. et al. (2020). Transformers: State-of-the-Art NLP. EMNLP 2020.
7. Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. ACM Multimedia 2015.
8. PyTorch Documentation. <https://pytorch.org/docs/stable/>
9. Hugging Face Transformers. <https://huggingface.co/docs/transformers>
10. Weights & Biases. <https://wandb.ai> — W&B Project: DL-genai-26-t1